



Algorytmy i Struktury Danych

Projekt: Optymalizacja planu zajęć

Projekt grupowy (6–7 osób)

Opracowali: dr inż. Sławomir Ziętek, dr Krzysztof Grochot

2025/2026, semestr zimowy

Tematem projektu jest zaprojektowanie i implementacja systemu do automatycznego układania planu zajęć na uczelni wyższej. System musi uwzględniać ograniczenia twarde (brak kolizji, dostępność sal, zgodność typów) oraz optymalizować ograniczenia miękkie (preferencje prowadzących analizowane przez model językowy LLM). Projekt realizowany jest w zespołach 6–7 osobowych i kończy się prezentacją połączoną z konkursem między grupami.

1. Cel projektu i kontekst

Problem układania planu zajęć (ang. timetabling problem) należy do klasy problemów NP-trudnych. Jego rozwiązanie w sposób optymalny dla realnych danych wymaga zastosowania zaawansowanych algorytmów przeszukiwania przestrzeni rozwiązań. Celem projektu jest nie tylko znalezienie poprawnego planu, lecz również jego optymalizacja pod kątem komfortu prowadzących i efektywności wykorzystania zasobów.

Po zakończeniu projektu studenci będą potrafili:

- zaprojektować reprezentację problemu kombinatorycznego jako graf lub model ograniczeń,
- zaimplementować od podstaw algorytm optymalizacji (bez użycia gotowych solverów),
- zintegrować model językowy (LLM) z systemem decyzyjnym,
- ocenić i porównać jakość różnych strategii heurystycznych,
- zaprezentować złożony system informatyczny w formie czytelnej wizualizacji.

2. Opis problemu — dane wejściowe

System operuje na syntetycznym zestawie danych uczelnianych obejmującym trzy kategorie obiektów. Dane dostarczane są w formacie JSON opisanym poniżej.

2.1. Prowadzący (~30 osób)

Każdy prowadzący opisany jest przez:

- unikalny identyfikator i imię/nazwisko,
- listę przedmiotów, których może nauczać,
- preferencje opisane tekstem naturalnym (np. „Wolę prowadzić zajęcia rano, najlepiej we wtorek i czwartek. Nie prowadzę w piątek po południu.”) — model LLM automatycznie ekstrahuje z tego tekstu dostępność i preferowane sloty,
- maksymalną liczbę godzin zajęć tygodniowo.

2.2. Sale (~50 pomieszczeń)

Każda sala opisana jest przez:

- unikalny identyfikator i nazwę,
- typ: aula wykładowa, sala ćwiczeniowa, laboratorium komputerowe, laboratorium sprzętowe,
- pojemność (liczba miejsc),
- dostępność (lista slotów, w których sala jest wolna).

2.3. Przedmioty i zajęcia (~60 przedmiotów)

Każdy przedmiot definiuje:

- nazwę i kierunek studiów,
- typ zajęć: wykład (W), laboratorium (L), ćwiczenia (C), projekt (P),
- liczbę studentów w grupie,
- wymaganą liczbę godzin tygodniowo,
- preferowany typ sali (np. laboratorium komputerowe dla zajęć programistycznych).

2.4. Format pliku JSON

```
{
  "instructors": [
    { "id": "I01", "name": "dr Kowalski",
      "subjects": ["MAT1", "AiSD"],
      "preferences_text": "Preferuję zajęcia rano. Nie prowadzę w piątek.",
      "max_hours_per_week": 16 }
  ],
  "rooms": [
    { "id": "A101", "type": "lecture_hall", "capacity": 120,
      "availability": {"Mon": [8,9,10,11,12,13,14,15,16]} }
  ],
  "courses": [
    { "id": "AiSD", "name": "Algorytmy i Struktury Danych",
      "type": "lab", "students": 25, "hours_per_week": 2,
      "required_room_type": "computer_lab" }
  ]
}
```

Dane wejściowe: Generator syntetycznych danych jest udostępniony przez prowadzącego. Prowadzący podaje platformę (UPEL lub Teams), z której należy skorzystać. Każdy zespół otrzymuje ten sam zestaw bazowy, ale może go rozszerzać.

3. Ograniczenia — definicja problemu

3.1. Ograniczenia twarde (Hard constraints)

Ograniczenia twarde muszą być bezwzględnie spełnione. Plan zawierający choćby jedno naruszenie jest traktowany jako niepoprawny i dyskwalifikuje zespół z rankingu konkursowego.

- HC-1 Brak kolizji prowadzącego: jeden prowadzący nie może mieć dwóch zajęć w tym samym slocie czasowym.
- HC-2 Brak kolizji sali: jedna sala nie może być zarezerwowana przez dwa różne zajęcia w tym samym slocie.
- HC-3 Brak kolizji grupy studenckiej: studenci jednej grupy nie mogą mieć dwóch zajęć równocześnie.
- HC-4 Dostępność prowadzącego: zajęcia mogą być planowane wyłącznie w slotach wyznaczonych na podstawie preferencji tekstowych wyekstrahowanych przez model LLM z pola „preferences_text”.
- HC-5 Dostępność sali: zajęcia mogą odbywać się wyłącznie w slotach dostępności danej sali.
- HC-6 Zgodność typu sali: typ sali musi odpowiadać typowi zajęć (wykład → aula, laboratorium → lab. komputerowe lub sprzętowe, ćwiczenia/projekt → sala ćwiczeniowa lub lab.).
- HC-7 Pojemność sali: liczba miejsc w sali $\geq$ liczba studentów w grupie.
- HC-8 Kompetencje prowadzącego: prowadzący może nauczać wyłącznie przedmiotów z własnej listy kompetencji.

3.2. Ograniczenia miękkie (Soft constraints)

Ograniczenia miękkie są optymalizowane — im więcej jest spełnionych, tym wyższa jakość planu. Każde naruszenie powoduje zmniejszenie wyniku funkcji celu.

- SC-1 Preferencje prowadzących (ekstraowane przez LLM): zajęcia planowane w preferowanych dniach/godzinach — waga: wysoka.
- SC-2 Minimalizacja okienek dla prowadzących: prowadzący nie powinien mieć długich przerw między zajęciami w obrębie jednego dnia — waga: średnia.
- SC-3 Grupowanie zajęć tego samego przedmiotu: wykład i laboratorium tego samego przedmiotu powinny przypadać na różne dni tygodnia — waga: średnia.
- SC-4 Równomierne rozłożenie obciążenia: zajęcia powinny być równomiernie rozłożone na wszystkie dni tygodnia — waga: niska.
- SC-5 Minimalizacja przemieszczania się studentów: kolejne zajęcia grupy studenckiej powinny odbywać się w bliskich lokalizacyjnie salach — waga: niska.
- SC-6 Nieprzekraczanie maksymalnego obciążenia tygodniowego prowadzącego — waga: wysoka.

4. Architektura systemu — wymagane moduły

System musi składać się z czterech niezależnych, ale współpracujących modułów. Każdy moduł powinien być odpowiedzialny za ściśle określoną funkcjonalność i posiadać jasny interfejs wymiany danych z pozostałymi modułami.

Moduł 1: Parser i walidator danych wejściowych

Odpowiada za wczytanie danych z pliku JSON, ich walidację (kompletność, spójność typów, zakresy wartości) oraz konwersję do wewnętrznej struktury danych używanej przez silnik optymalizacji. W przypadku błędów w danych wejściowych moduł powinien generować czytelne komunikaty o błędach.

Moduł 2: Silnik optymalizacji

Główna część projektu. Musi zawierać własną (od podstaw) implementację pary algorytmów optymalizacyjnych: algorytmu konstruktywnego (bazowego) oraz algorytmu poprawiającego. Zalecane jest samodzielne przebadanie dostępnej literatury i dokonanie świadomego wyboru podejścia — różne metody mają odmienne właściwości w kontekście problemów harmonogramowania i warto to uwzględnić przy wyborze.

Ważne: Korzystanie z gotowych solwerów CSP/ILP (OR-Tools, PuLP, CPLEX, Gurobi, Z3 itp.) jest niedozwolone — każdy algorytm powinien być zaimplementowany samodzielnie. W razie wątpliwości co do klasyfikacji danego narzędzia warto skonsultować się z prowadzącym przed przystąpieniem do implementacji.

Silnik musi implementować:

- funkcję celu (fitting function) liczącą łączną liczbę naruszeń ograniczeń miękkich z uwzględnieniem wag,
- weryfikator ograniczeń twardych,
- logowanie postępu optymalizacji (wartość funkcji celu w kolejnych iteracjach),
- pomiar czasu działania algorytmu.

Moduł 3: Analiza preferencji przez LLM

Moduł komunikuje się z modelem językowym (LLM) w celu ekstrakcji ustrukturyzowanych preferencji prowadzącego z tekstu naturalnego. Wejściem jest pole "preferences_text" z danych JSON, wyjściem — ustrukturyzowany obiekt z preferencjami czasowymi gotowy do użycia przez silnik optymalizacji.

Przykład wejścia dla LLM:

Tekst: "Wolę wykłady rano, najlepiej we wtorek i czwartek między 8 a 12. Absolutnie nie prowadzę zajęć w piątek po południu. Laboratorium mogę mieć o dowolnej porze, ale nie chcę zaczynać przed 9:00."

Oczekiwane wyjście JSON:

```
{
  "preferred_days": ["Tue", "Thu"],
  "preferred_hours_start": 8,
  "preferred_hours_end": 12,
```

```
"forbidden_slots": [{"day": "Fri", "from": 13, "to": 20}],  
"min_start_hour": 9  
}
```

Konkurs — model LLM: W ramach konkursu system będzie testowany z modelem Bielik 11b uruchomionym na infrastrukturze uczelni (serwer lokalny, interfejs OpenAI-compatible API). Zalecane jest testowanie z różnymi modelami (GPT-4o, Llama, Mistral) i porównanie jakości ekstrakcji.

Moduł powinien zawierać:

- szablon promptu (system prompt + user prompt) z opisem zadania i formatem JSON,
- walidację odpowiedzi LLM (czy zwrócony JSON jest poprawny i kompletny),
- obsługę błędów (retry, fallback na domyślne preferencje),
- możliwość pracy offline (tryb bez LLM, preferencje ignorowane lub domyślne).

Moduł 4: Wizualizacja i raportowanie

Moduł prezentuje wyniki optymalizacji w czytelnej, atrakcyjnej formie. Wymagane są co najmniej trzy rodzaje widoków:

- Widok siatki planu zajęć: klasyczny tygodniowy plan w układzie dni × sloty godzinowe, z oznaczeniem prowadzącego, sali, przedmiotu i typu zajęć. Możliwość filtrowania według prowadzącego, sali lub grupy.
- Wykres postępu optymalizacji: wartość funkcji celu (liczba naruszeń SC) w kolejnych iteracjach algorytmu — osobno dla algorytmu A i B (porównanie).
- Raport statystyczny: procentowe zaspokojenie każdego ograniczenia miękkiego, obciążenie prowadzących (wykres słupkowy), wykorzystanie sal (mapa ciepła godziny × sale).

Technologia wizualizacji: Wizualizacja może być wykonana w dowolnej technologii: matplotlib, Plotly, Dash, Streamlit, JavaFX, React, Qt, itp. Ocenie podlega czytelność, kompletność i estetyka prezentacji danych, a nie wybór technologii.

5. Kryteria oceny

Ocena końcowa uwzględnia kilka wymiarów jakości dostarczonego rozwiązania. Szczegółowe zasady oraz ewentualne wagi poszczególnych kryteriów zostaną podane przez prowadzącego w trakcie semestru.

5.1. Ogólne kryteria oceny

- **Poprawność:** wygenerowany plan musi spełniać wszystkie zdefiniowane ograniczenia twarde (HC). Plan zawierający kolizje lub inne naruszenia HC jest traktowany jako niepoprawny.
- **Szybkość:** czas potrzebny do wygenerowania poprawnego planu ma znaczenie praktyczne — system powinien działać w rozsądnym czasie na typowym sprzęcie.
- **Jakość optymalizacji:** mierzona jako stopień zaspokojenia ograniczeń miękkich (SC-1..SC-6) z uwzględnieniem ich wag. Im wyższy procent spełnionych preferencji, tym lepsza ocena jakościowa rozwiązania.
- **Wizualizacja i raportowanie:** czytelna prezentacja planu oraz wykresu zbieżności algorytmu jest integralną częścią projektu i podlega osobnej ocenie.

6. Konkurs między grupami

Na zajęciach końcowych wszystkie grupy zmierzają się w rankingu jakości wygenerowanego planu. Szczegółowe zasady konkursu — w tym format danych testowych, sposób pomiaru i podział premii — zostaną podane przez prowadzącego w trakcie semestru.

Konkurs: Premia konkursowa wynosi 10 pkt i wlicza się do oceny końcowej. Warunkiem udziału jest plan spełniający wszystkie ograniczenia twarde (HC = 0).

7. Organizacja pracy zespołu

7.1. Role w zespole

- Menedżer projektu (1 osoba): koordynuje pracę, prowadzi backlog zadań, odpowiada za integrację modułów i terminowość. Oceniany jest m.in. za jakość dokumentacji i organizację pracy zespołu.
- Inżynier algorytmów (2–3 osoby): odpowiada za moduł optymalizacji (Zadania 3–6).
- Inżynier danych/LLM (1–2 osoby): odpowiada za parser danych i moduł LLM (Zadania 2, 7).
- Inżynier wizualizacji (1–2 osoby): odpowiada za moduł wizualizacji i raportowania (Zadanie 8).

8. Format oddania i wymagania formalne

Na ostatnich zajęciach każda grupa przeprowadza prezentację trwającą maksymalnie 10 minut + 5 minut pytań. Prezentacja musi zawierać demonstrację działającego systemu na żywo.

Do oceny wymagane jest dostarczenie:

- Kompletny kod źródłowy przesłany przez wskazaną przez prowadzącego platformę (UPEL lub Teams) najpóźniej na 24h przed prezentacją.
- Plik PDF ze sprawozdaniem końcowym (max 15 stron) zawierającym: opis zastosowanych algorytmów, analizę wyników, wnioski, wykres zbieżności.
- Slajdy prezentacji (PDF lub PowerPoint).
- Wyniki na danych testowych: plik JSON z wygenerowanym planem zajęć oraz log z wartościami funkcji celu.

Uwaga: Projekty oddane po terminie lub bez działającego demo tracą 20% punktów.

Wymagania dla wszystkich wykresów i wizualizacji:

- Wszystkie wykresy muszą mieć tytuł, opisy osi z jednostkami i legendę.
- Etykiety czytelne (minimum 10pt), kolorystyka czytelna w druku czarno-białym.
- Wykresy w sprawozdaniu w rozdzielczości minimum 150 DPI.